

## ATIVIDADE 2 - LABORATÓRIO DE REDES DE COMPUTADORES

NOMES: Gustavo de Assis & Arthur Bracarense

**1) Implementar o código da aplicação cliente com UDP, que envia uma linha de texto para o servidor.**

Tomando o código base fornecido, e configurando o IP nele, temos:

```
4 class UDPServer {
5     public static void main(String args[]) throws Exception
6     {
7         // Criação do socket UDP no servidor, indicando a porta
8         DatagramSocket serverSocket = new DatagramSocket(port:9876);
9         // Criação dos vetores de bytes para receber e enviar dados do/para o cliente
10        byte[] receiveData = new byte[1024];
11        byte[] sendData = new byte[1024];
12        // Loop infinito para executar o código no servidor
13        while(true)
14        {
15            // Criação da variável para receber o pacote de requisição do cliente
16            DatagramPacket receivePacket = new DatagramPacket (receiveData,
17            receiveData.length);
18            // Recebimento do pacote do cliente
19            serverSocket.receive(receivePacket);
20            System.out.println(x:"Datagrama UDP recebido.");
21            // Tratamento dos dados recebidos do cliente: sentença, IP e porta do cliente
22            String sentence = new String(receivePacket.getData());
23            InetAddress IPAddress = receivePacket.getAddress();
24            int port = receivePacket.getPort();
25            // Conversão da sentença para letras maiúsculas
26            String capitalizedSentence = sentence.toUpperCase();
27            // Conversão da sentença para bytes
28            sendData = capitalizedSentence.getBytes();
29            // Criação do pacote de resposta para o cliente
30            DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length,
31            IPAddress, port);
32            // Envio do pacote de resposta para o cliente
33            serverSocket.send(sendPacket);
34        }
35    }
36 }
```

**2) Implementar o código da aplicação servidor com UDP, que recebe a linha de texto e transforma as letras minúsculas em maiúsculas e retorna a linha de texto para o cliente.**

No maravilhoso NetBeans :)

```
1 package com.mycompany.arthurbraca;
2
3 import java.net.*; // Biblioteca para trabalhar com sockets
4 import java.io.*; // Biblioteca para trabalhar com buffers
5
6 public class ArthurBraca {
7
8     public static void main(String args[]) throws Exception {
9         // Criação do socket UDP sem definir a porta
10        DatagramSocket clientSocket = new DatagramSocket();
11        // Obtenção do IP do servidor: no código ou pelo usuário
12        //InetAddress IPAddress = InetAddress.getByName("172.22.0.71");
13        System.out.println("Digite o IP do servidor UDP:");
14        BufferedReader inFromUser = new BufferedReader(new InputStreamReader(System.in));
15        InetAddress IPAddress = InetAddress.getByName(inFromUser.readLine());
16        // Criação dos vetores de bytes para enviar e receber dados para/do o servidor
17        byte[] sendData = new byte[1024];
18        byte[] receiveData = new byte[1024];
19        // Obtenção da sentença para enviar ao servidor
20        System.out.println("Digite o texto a ser enviado para o servidor UDP:");
21        String sentence = inFromUser.readLine();
22        // Conversão da sentença para bytes
23        sendData = sentence.getBytes();
24        // Criação do pacote de envio ao servidor
25        DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length,
26        IPAddress, 9876);
27        // Envio do pacote ao servidor
```

3) Executar os códigos cliente e servidor em máquinas distintas e realizar testes. Tirar print da tela do cliente com o envio e a resposta.

Servidor:

```
Experimente: apt install <deb name>
aluno@5A445JW93:~/Área de trabalho/Pratica02-LRDC-ArthurBracarense/src/main/java/pratica02/lrdc/arthurbracarense$ java Pratica02LRDCArthurBracarense.java
Datagrama UDP recebido.
Datagrama UDP recebido.
```

Cliente:

```
Digite o IP do servidor UDP: 172.22.0.73
Digite o texto a ser enviado para o servidor UDP:
TESTE
RESPOSTA DO SERVIDOR UDP:TESTE
```

4) Implementar o código da aplicação cliente com TCP, que envia uma linha de texto para o servidor.

5) Implementar o código da aplicação servidor com TCP, que recebe a linha de texto e transforma as letras minúsculas em maiúsculas e retorna a linha de texto para o cliente.

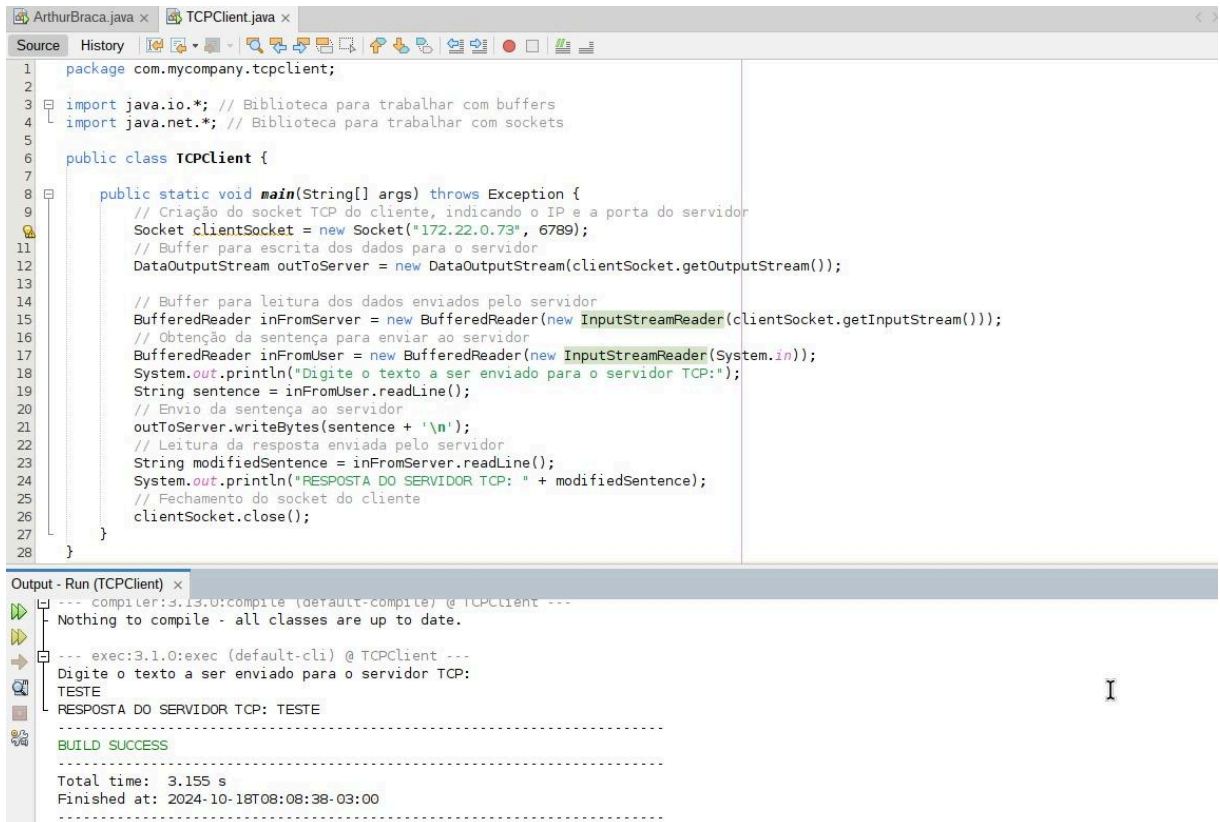
6) Executar os códigos cliente e servidor em máquinas distintas e realizar testes. Tirar print da tela do cliente com o envio e a resposta.

Servidor:

```
J Pratica02LRDCArthurBracarense.java x JS server.js Welcome
src > main > java > pratica02 > lrdc > arthurbracarense > J Pratica02LRDCArthurBracarense.java > TCPServer > main(String[])
1 package pratica02.lrdc.arthurbracarense;
2
3 import java.io.*; // Biblioteca para trabalhar com buffers
4 import java.net.*; // Biblioteca para trabalhar com sockets
5 class TCPServer {
6     Run | Debug
7     public static void main(String argv[]) throws Exception
8     {
9         // Criação do socket de conexão TCP no servidor, indicando a porta
10        ServerSocket welcomeSocket = new ServerSocket(port:6789);
11
12        while(true) {
13            // Criação do socket de dados a partir do estabelecimento de uma conexão
14            Socket connectionSocket = welcomeSocket.accept();
15            System.out.println(x:"Conexão TCP recebida.");
16            // Buffer para leitura dos dados enviados pelo cliente
17            BufferedReader inFromClient = new BufferedReader(new
18            InputStreamReader(connectionSocket.getInputStream()));
19            // Buffer para escrita dos dados para o cliente
20            DataOutputStream outToClient = new
21            DataOutputStream(connectionSocket.getOutputStream());
22            // Leitura dos dados enviados pelo cliente
23            String clientSentence = inFromClient.readLine();
24            System.out.println(clientSentence);
25            // Conversão da sentença para letras maiúsculas
26            String capitalizedSentence = clientSentence.toUpperCase() + '\n';
27            // Envio da resposta ao cliente
28            outToClient.writeBytes(capitalizedSentence);
29        }
30    }
31 }

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
aluno@5A445JW93:~/Área de trabalho/Pratica02-LRDC-ArthurBracarense/src/main/java/pratica02/lrdc/arthurbracarense$ java Pratica02LRDCArthurBracarense.java
Conexão TCP recebida.
TESTE
```

Cliente:



```
1 package com.mycompany.tcpclient;
2
3 import java.io.*; // Biblioteca para trabalhar com buffers
4 import java.net.*; // Biblioteca para trabalhar com sockets
5
6 public class TCPClient {
7
8     public static void main(String[] args) throws Exception {
9         // Criação do socket TCP do cliente, indicando o IP e a porta do servidor
10        Socket clientSocket = new Socket("172.22.0.73", 6789);
11        // Buffer para escrita dos dados para o servidor
12        DataOutputStream outToServer = new DataOutputStream(clientSocket.getOutputStream());
13
14        // Buffer para leitura dos dados enviados pelo servidor
15        BufferedReader inFromServer = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));
16        // Obtenção da sentença para enviar ao servidor
17        BufferedReader inFromUser = new BufferedReader(new InputStreamReader(System.in));
18        System.out.println("Digite o texto a ser enviado para o servidor TCP:");
19        String sentence = inFromUser.readLine();
20        // Envio da sentença ao servidor
21        outToServer.writeBytes(sentence + '\n');
22        // Leitura da resposta enviada pelo servidor
23        String modifiedSentence = inFromServer.readLine();
24        System.out.println("RESPOSTA DO SERVIDOR TCP: " + modifiedSentence);
25        // Fechamento do socket do cliente
26        clientSocket.close();
27    }
28 }
```

Output - Run (TCPClient) x

```
--- compiler:3.13.0:compile (default-compile) @ TCPClient ---
Nothing to compile - all classes are up to date.
--- exec:3.1.0:exec (default-cli) @ TCPClient ---
Digite o texto a ser enviado para o servidor TCP:
TESTE
RESPOSTA DO SERVIDOR TCP: TESTE
BUILD SUCCESS
Total time: 3.155 s
Finished at: 2024-10-18T08:08:38-03:00
```

O exemplo que usei não foi dos melhores kkk, já que já enviei a mensagem em letras maiúsculas, mas ele faz isso.

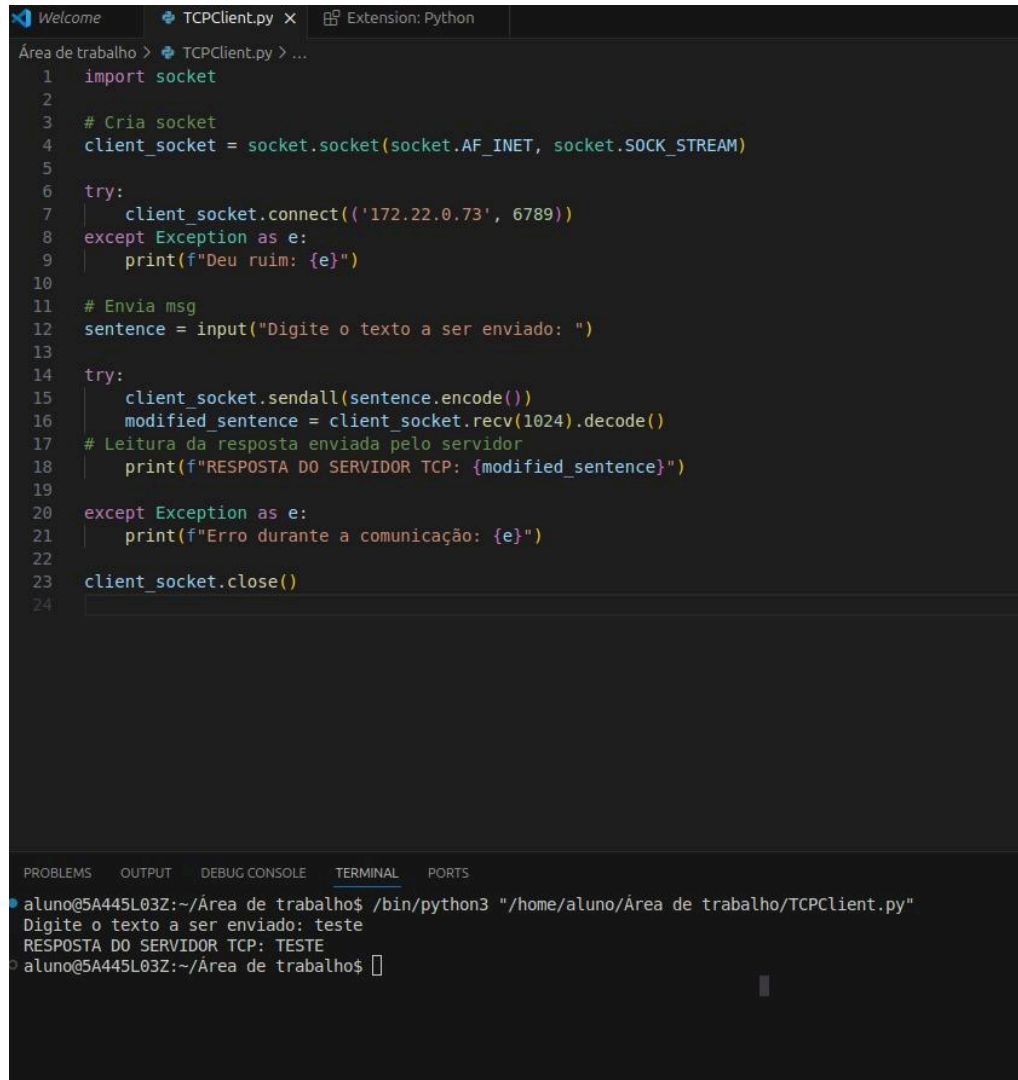
**7) Implementar os códigos cliente e servidor com TCP em alguma outra linguagem, por exemplo Python. Enviar os códigos e o print das telas do cliente e do servidor em formato PDF no SIGAA.**



```
1 import socket
2
3 # Criação do socket de conexão TCP no servidor
4 server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5 server_socket.bind(('0.0.0.0', 6789))
6 server_socket.listen(1)
7
8 print("Servidor TCP aguardando conexões...")
9
10 while True:
11     # Criação do socket de dados
12     connection_socket, addr = server_socket.accept()
13     print(f"Conexão TCP recebida de {addr}")
14
15     try:
16         # Leitura dos dados enviados pelo cliente
17         client_data = connection_socket.recv(1024).decode()
18         if not client_data:
19             continue
20
21         capitalized_sentence = client_data.upper()
22         connection_socket.sendall(capitalized_sentence.encode())
23
24     except Exception as e:
25         print(f"Erro durante a comunicação: {e}")
26
27     finally:
28         # Fechamento do socket de conexão com o cliente
29         connection_socket.close()
```

```
aluno@5A445JW93:~/Área de trabalho/Pratica02-LRDC-ArthurBracarense/tcpJavaScript$ python3 server.py
Servidor TCP aguardando conexões...
Conexão TCP recebida de ('172.22.0.71', 59482)
```

Cliente:



The image shows a VS Code editor window with a file named `TCPClient.py` open. The code is a Python script that connects to a server at `172.22.0.73` on port `6789`. It sends a message and receives a response. The terminal at the bottom shows the execution of the script, where the user enters the text `teste` and receives the response `TESTE` from the server.

```
1 import socket
2
3 # Cria socket
4 client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5
6 try:
7     client_socket.connect(('172.22.0.73', 6789))
8 except Exception as e:
9     print(f"Deu ruim: {e}")
10
11 # Envia msg
12 sentence = input("Digite o texto a ser enviado: ")
13
14 try:
15     client_socket.sendall(sentence.encode())
16     modified_sentence = client_socket.recv(1024).decode()
17 # Leitura da resposta enviada pelo servidor
18     print(f"RESPOSTA DO SERVIDOR TCP: {modified_sentence}")
19
20 except Exception as e:
21     print(f"Erro durante a comunicação: {e}")
22
23 client_socket.close()
24
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
aluno@5A445L03Z:~/Área de trabalho$ /bin/python3 "/home/aluno/Área de trabalho/TCPClient.py"
Digite o texto a ser enviado: teste
RESPOSTA DO SERVIDOR TCP: TESTE
aluno@5A445L03Z:~/Área de trabalho$
```